

Deep Q-Learning and Planning in Self-Play Mini-Chess: A Comparison of DQN, Double DQN, and MCTS

Zhenkang Dong
Northeastern University
Boston, Massachusetts 02115
dong.zhenk@northeastern.edu

Abstract

We study the application of value-based deep reinforcement learning to two-player adversarial board games by training self-play agents in 5×5 Gardner Mini-Chess. Three methods are compared under controlled conditions: Deep Q-Network (DQN), Double DQN, and Monte Carlo Tree Search (MCTS) with random rollouts. DQN and Double DQN share identical network architectures, hyperparameters, and training budgets, isolating the effect of the target-estimation mechanism. Agents are trained via frozen-opponent self-play and evaluated against fixed random and heuristic opponents. We find that MCTS, requiring no training, achieves a 93% win rate against the random opponent, far exceeding both DQN (13%) and Double DQN (18%). Double DQN exhibits lower Q-value overestimation (mean $Q = 1.84$ vs. 1.89) and lower cross-seed variance ($\pm 0.8\%$ vs. $\pm 1.4\%$) than standard DQN, confirming the theoretical prediction. Both learned agents struggle with the sparse terminal reward structure, frequently drawing rather than winning. We discuss the implications of these findings for applying value-based RL to adversarial games and identify reward shaping as a promising direction for improvement.

Introduction

Board games have long served as benchmark problems for artificial intelligence, from Samuel’s checkers player (Samuel 1959) to Deep Blue (Campbell, Hoane Jr, and Hsu 2002) and AlphaGo (Silver et al. 2016). More recently, AlphaZero (Silver et al. 2018) demonstrated that a combination of Monte Carlo Tree Search and deep neural networks can achieve superhuman play across chess, shogi, and Go, all from self-play alone. These results are impressive, but the computational resources involved—thousands of TPUs training for millions of games—place them beyond the reach of most researchers.

A natural question is whether simpler reinforcement learning methods can learn meaningful strategies in smaller-scale adversarial games. Deep Q-Networks (DQN) (Mnih et al. 2015) have been enormously successful in single-agent settings such as Atari, but their application to two-player games introduces several complications: the environment is no longer stationary (the opponent adapts), rewards are typ-

ically sparse and terminal, and the action space may require masking for legality.

This project investigates these challenges in a controlled setting. We use 5×5 Gardner Mini-Chess (Gardner 1970), a variant of chess played on a reduced board with 10 pieces per side. Despite its smaller size, this game retains the essential strategic elements of standard chess—tactics, piece coordination, pawn promotion—and has been weakly solved with a game-theoretic value of draw (Ehsan and Hashemi 2012). The reduced state and action spaces make it feasible to train DQN on a single consumer GPU, while remaining complex enough that a random agent wins only a small fraction of games against any structured opponent.

We compare three methods:

1. **DQN** (Mnih et al. 2015), which serves as the baseline. We adapt DQN to the two-player setting via frozen-opponent self-play and legal-action masking.
2. **Double DQN** (Van Hasselt, Guez, and Silver 2016), which uses the identical architecture and hyperparameters as DQN but modifies the target computation to reduce overestimation bias. By holding everything else fixed, we isolate the effect of this single algorithmic change.
3. **MCTS** (Coulom 2006; Kocsis and Szepesvári 2006), which is a planning-based method using UCB1 selection and random rollouts. It requires no training and serves as a non-learning reference point.

The central question is not which method achieves the highest playing strength—MCTS with sufficient simulations will naturally dominate—but rather how DQN and Double DQN compare *as learning algorithms* in terms of convergence behavior, value estimation stability, and cross-seed variance.

Our main findings are: (1) MCTS strongly outperforms both learned agents, highlighting the difficulty of pure value learning under sparse rewards; (2) Double DQN produces lower Q-value estimates and smaller overestimation bias than standard DQN, consistent with the theory of van Hasselt et al. (Van Hasselt, Guez, and Silver 2016); (3) both DQN variants converge to predominantly drawing strategies, a behavior we attribute to the combination of sparse terminal rewards and discounted value estimation.

Background

Markov Decision Processes

We model the agent-environment interaction as a Markov Decision Process (MDP) defined by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s' | s, a)$ is the transition function, $R(s, a)$ is the reward function, and $\gamma \in [0, 1]$ is a discount factor. The agent seeks a policy $\pi(a | s)$ that maximizes the expected cumulative discounted reward $\mathbb{E}_\pi [\sum_{t=0}^{\infty} \gamma^t r_t]$.

In the two-player setting, the environment includes the opponent’s behavior. When the opponent’s policy is fixed, the problem reduces to a standard (single-agent) MDP from the learning agent’s perspective, though the transition dynamics are non-stationary if the opponent is periodically updated.

Deep Q-Networks

The action-value function $Q^\pi(s, a) = \mathbb{E}_\pi [\sum_{t=0}^{\infty} \gamma^t r_t | s_0 = s, a_0 = a]$ satisfies the Bellman equation:

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s' | s, a) \max_{a'} Q^*(s', a'). \quad (1)$$

DQN (Mnih et al. 2015) approximates Q^* with a neural network $Q(s, a; \theta)$ and minimizes the temporal-difference loss

$$L(\theta) = \mathbb{E} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right], \quad (2)$$

where θ^- denotes the parameters of a periodically updated target network. Two key ingredients stabilize learning: experience replay, which stores transitions (s, a, r, s') in a buffer and samples minibatches uniformly, and the target network, which provides a slowly changing regression target.

Double DQN

Van Hasselt et al. (Van Hasselt, Guez, and Silver 2016) observed that the max operator in Equation 2 uses the same network to both *select* and *evaluate* the next action, inducing an upward bias in Q-value estimates. Double DQN decouples these two operations:

$$y = r + \gamma Q\left(s', \arg \max_{a'} Q(s', a'; \theta); \theta^-\right). \quad (3)$$

The online network θ selects the greedy action, while the target network θ^- evaluates it. This simple change has been shown to reduce overestimation and improve stability in Atari benchmarks (Van Hasselt, Guez, and Silver 2016).

Monte Carlo Tree Search

MCTS (Coulom 2006; Kocsis and Szepesvári 2006) builds a search tree incrementally through repeated simulations. Each simulation consists of four phases: *selection* (traverse the tree using UCB1), *expansion* (add a new node), *rollout* (play random moves to a terminal state), and *backpropagation* (update visit counts and value estimates along the path).

The UCB1 selection rule balances exploitation and exploration:

$$\text{UCB1}(s, a) = \frac{W(s, a)}{N(s, a)} + c \sqrt{\frac{\ln N(s)}{N(s, a)}}, \quad (4)$$

where $W(s, a)$ is the cumulative value, $N(s, a)$ is the visit count, and $c = \sqrt{2}$ is the exploration constant. After all simulations, the action with the highest visit count at the root is selected.

Related Work

Deep RL for board games. The most prominent application of deep RL to board games is the AlphaGo/AlphaZero family (Silver et al. 2016, 2018), which combines MCTS with a policy-value network trained via self-play. These systems achieve superhuman performance but require substantial compute. Tian and Zhu (Tian and Zhu 2016) applied convolutional networks to Go with a simpler training pipeline, showing that deep features can capture positional understanding even without search. Lai (Lai 2015) trained a deep neural network to play standard chess from self-play, using a position evaluation approach rather than full MCTS integration.

DQN in adversarial settings. While DQN was originally developed for single-agent Atari games (Mnih et al. 2015), several works have explored its application to multi-agent and adversarial domains. Tampuu et al. (Tampuu et al. 2017) applied DQN to multi-agent Pong, demonstrating both cooperative and competitive behaviors. The key adaptation for two-player games is self-play, where the agent plays against a copy of itself. Heinrich and Silver (Heinrich and Silver 2016) studied deep RL in imperfect-information games, noting the difficulty of non-stationary opponent distributions.

Mini-chess and reduced board games. Gardner (Gardner 1970) proposed the 5×5 chess variant, and Ehsan and Hashemi (Ehsan and Hashemi 2012) weakly solved it, establishing the game-theoretic value as a draw. Bhatt (Bhatt 2018) used a reinforcement learning approach for a mini-chess variant, and the open-source implementation we build upon provides rule enforcement for several mini-chess variants. More broadly, reduced board games have been used as testbeds for RL algorithms when full-scale games are computationally prohibitive (Lanctot et al. 2019).

Methods

Gardner Mini-Chess Environment

Gardner Mini-Chess is played on a 5×5 board with the following initial configuration: each side has a King, Queen, Rook, Bishop, Knight, and five Pawns, arranged in the first two rows. Standard chess rules apply with the omission of castling and the double pawn push. Pawns promote upon reaching the back rank. The game terminates on checkmate (decisive result) or when a position is reached where the active player has no legal moves (stalemate, counted as draw). We impose a 200 half-move limit to prevent infinite games; any game reaching this limit is scored as a draw.

We build on an open-source Python implementation that handles rule enforcement, move generation, and game-state management. The environment is wrapped in a Gym-like interface that the RL agent interacts with.

State Representation

The board state is encoded as a $12 \times 5 \times 5$ binary tensor from the active player’s perspective (canonical encoding). Channels 0–5 represent the current player’s pieces {Pawn, Knight, Bishop, Rook, Queen, King}; channels 6–11 represent the opponent’s pieces in the same order. When it is Black’s turn, the board is rotated 180° and the piece colors are swapped, so the network always observes the game from the same viewpoint. This canonical representation allows a single network to play both colors.

Action Representation

The action space consists of $|\mathcal{A}| = 625$ discrete actions, corresponding to all from-square $\times$ to-square combinations (25×25). A legal-action mask $m \in \{0, 1\}^{625}$ is applied at each step: illegal actions receive $Q = -\infty$ before the arg max during action selection. Pawn promotion defaults to queen; underpromotion is handled by the environment when multiple promotion types share the same from-to square pair.

DQN Architecture and Training

Both DQN and Double DQN use the same convolutional architecture:

- Conv2d($12 \rightarrow 64, 3 \times 3, \text{padding}=1$) $\rightarrow$ BatchNorm $\rightarrow$ ReLU
- Conv2d($64 \rightarrow 128, 3 \times 3, \text{padding}=1$) $\rightarrow$ BatchNorm $\rightarrow$ ReLU
- Conv2d($128 \rightarrow 128, 3 \times 3, \text{padding}=1$) $\rightarrow$ BatchNorm $\rightarrow$ ReLU
- Flatten $\rightarrow$ FC($3200 \rightarrow 512$) $\rightarrow$ ReLU $\rightarrow$ FC($512 \rightarrow 625$)

The output layer has 625 units, one per action, representing Q-values. Training uses Adam (Kingma and Ba 2014) with learning rate 10^{-4} , minibatch size 256, replay buffer of capacity 10^5 transitions, and target network updates every 1000 optimization steps. Exploration follows a linear ε -greedy schedule from $\varepsilon = 1.0$ to $\varepsilon = 0.05$ over 30,000 episodes. The discount factor is $\gamma = 0.99$. We use Huber loss (smooth L_1) instead of MSE to improve robustness to large TD errors, and clamp target Q-values to $[-2, 2]$ to prevent divergence. Gradients are clipped at $\|\nabla\| \leq 1.0$.

The only difference between the DQN and Double DQN agents is the target computation: DQN uses Equation 2; Double DQN uses Equation 3.

Self-Play Training Procedure

The training procedure operates in two phases. During a warmup phase (episodes 1–1000), the agent plays against a purely random opponent. After warmup, the agent transitions to self-play against a *frozen* copy of its own network, which is refreshed every 500 episodes. This frozen-opponent

approach, related to fictitious self-play (Heinrich and Silver 2016), provides a more stable training signal than playing against the constantly updating online network, since it keeps the effective transition dynamics approximately stationary between updates.

To mitigate first-move advantage, the agent alternates between playing White and Black on successive episodes. The canonical state encoding ensures the network can play both sides without modification.

MCTS Implementation

We implement a standard MCTS with UCB1 selection (Equation 4) and uniform random rollouts. At each decision point, 200 simulations are performed. Each simulation consists of selection, expansion of one child node, a random rollout to termination (or 200-move depth limit), and back-propagation of the result. Values are stored from White’s perspective at each node, and the selection formula is adjusted depending on the active player’s color. After all simulations, the root action with the most visits is selected.

Evaluation Opponents

We evaluate against two fixed opponents:

- **Random:** selects uniformly among legal moves.
- **Heuristic:** prioritizes checking moves; if none exist, selects a random capture; otherwise plays a random legal move.

The heuristic opponent is stronger than random because it exploits immediate tactical opportunities, but it has no strategic understanding.

Experiments

Experimental Setup

All DQN and Double DQN experiments are run for 20,000 episodes with two random seeds (42 and 123). Evaluation is conducted every 2,000 episodes by playing 30 games against each fixed opponent, with colors alternating. MCTS is evaluated separately by playing 30 games against each opponent with 200 simulations per move. All experiments are run on a single NVIDIA RTX 4060 Laptop GPU.

Training Dynamics

Figure 1 shows the training win rate over 20,000 episodes. Both algorithms exhibit a similar trajectory: an initial rise during the warmup phase (against a random opponent), followed by a decline and stabilization after the transition to self-play at episode 1,000. The convergent win rate of approximately 15% reflects the difficulty of winning against a gradually improving frozen opponent—a high fraction of games end in draws. The two methods produce comparable learning curves, with Double DQN showing marginally lower variance across seeds (visible in the shaded regions).

The training loss (Figure 2) converges to approximately 0.003 for both methods. The loss increases early in training as the agent’s experiences become more diverse, then stabilizes. We note that an earlier version of our implementation

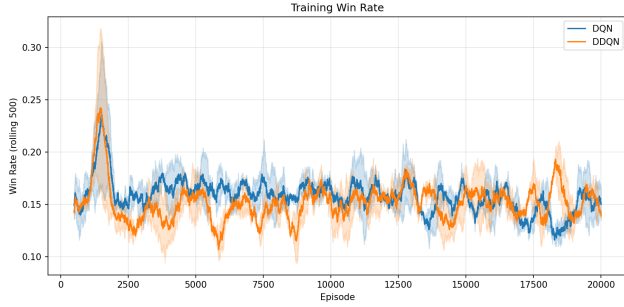


Figure 1: Training win rate (rolling average over 500 episodes) for DQN and Double DQN in self-play. Shaded regions indicate ± 1 standard deviation across seeds.

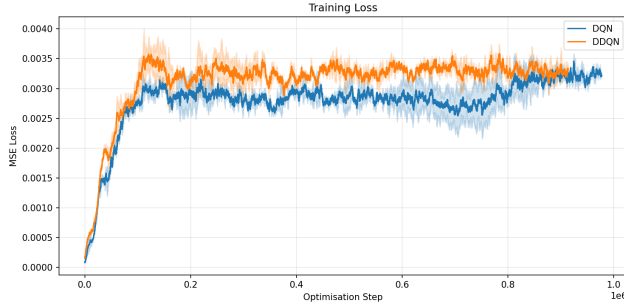


Figure 2: Training loss (Huber) over optimization steps.

using MSE loss without Q-value clamping exhibited catastrophic loss divergence (reaching 10^6); the switch to Huber loss and target clamping resolved this.

Q-Value Overestimation Analysis

The overestimation analysis (Figure 3) is the central comparison of this project. In the converged regime (last 20% of training), DQN produces an average Q-value of 1.891 (± 0.001 across seeds), while Double DQN produces 1.841 (± 0.008). The overestimation bias—defined as $\bar{Q}(s, a) - \bar{y}$, where $\bar{y}$ is the Bellman target—is 6.96×10^{-4} for DQN and 6.16×10^{-4} for Double DQN (Table 1).

Although both biases are small in absolute magnitude (a consequence of the Q-value clamping to $[-2, 2]$), DQN’s overestimation is consistently about 13% larger than Double DQN’s, and this difference is persistent throughout training (Figure 3b). This confirms the theoretical prediction of van Hasselt et al. (Van Hasselt, Guez, and Silver 2016): the max operator in standard DQN introduces an upward bias that Double DQN mitigates by decoupling action selection from value evaluation.

Evaluation Against Fixed Opponents

Table 2 and Figure 4 present the evaluation results. MCTS achieves a 93% win rate against the random opponent and 63% against the heuristic opponent, while never losing a single game to the random player. Both DQN and Double DQN achieve modest win rates (13% and 18%, respectively)

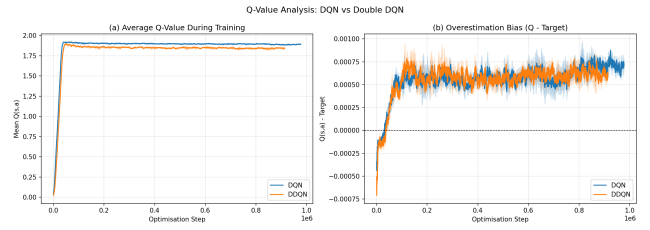


Figure 3: Q-value analysis: (a) mean Q-value during training; (b) overestimation bias ($Q - \text{target}$). Both are smoothed over a 2000-step window.

Table 1: Q-value statistics in the converged regime (last 20% of training), averaged across two seeds.

	DQN	Double DQN
Mean $Q(s, a)$	1.891	1.841
Mean target y	1.890	1.840
Overest. bias ($\times 10^{-4}$)	6.96	6.16
Cross-seed win-rate std	$\pm 1.4\%$	$\pm 0.8\%$

against random, with the majority of games ending in draws. Against the heuristic opponent, neither DQN nor Double DQN wins more than 5% of games.

The dominance of MCTS is expected: it performs explicit lookahead search at each decision point, which is extremely effective in a deterministic, fully observable game. The more interesting observation is the behavior of the DQN variants. Both agents have learned to *avoid losing*—their loss rates are low—but they cannot reliably find *winning* strategies. The result is a conservative, draw-prone style of play.

Discussion

Several factors contribute to the limited playing strength of DQN and Double DQN:

Sparsely terminal rewards. The agent receives a reward signal only at the end of the game, which may be 40–50 moves away. Under $\gamma = 0.99$, a reward 50 steps into the future is discounted by a factor of $0.99^{50} \approx 0.61$. The network must propagate this weak signal backward through many transitions via bootstrapping, which is slow and noisy. In contrast, MCTS evaluates positions by directly simulating games to completion, bypassing the credit-assignment problem entirely.

Self-play non-stationarity. Although the frozen-opponent approach mitigates the worst forms of non-stationarity, the opponent distribution still shifts every 500 episodes. This means the agent repeatedly faces a new effective MDP, and value estimates learned against one opponent version may be inaccurate against the next.

Draw incentive structure. With terminal rewards of +1, −1, and 0, a risk-averse policy that avoids losing (and thereby draws) is only slightly worse than a policy that sometimes wins but sometimes loses. The value function tends to assign similar values to many states, making it difficult for the agent to differentiate winning from drawing lines

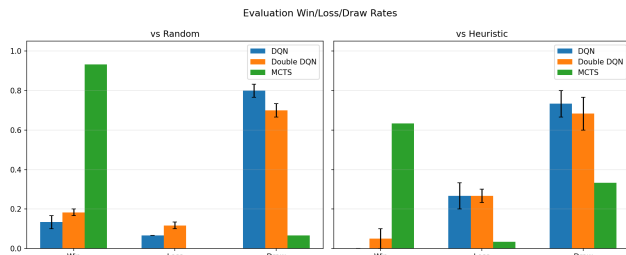


Figure 4: Win / loss / draw rates against random and heuristic opponents. DQN and Double DQN results are from the final evaluation checkpoint (episode 20,000), averaged across seeds. Error bars show ± 1 std.

Table 2: Evaluation results (win / loss / draw %) at episode 20,000, averaged across two seeds. MCTS uses 200 simulations per move.

Opponent	DQN	DDQN	MCTS
vs.Random (W/L/D)	13/7/80	18/12/70	93/0/7
vs.Heuristic (W/L/D)	0/27/73	5/27/68	63/3/33

of play.

Comparison with related work. These challenges are well-known in the RL literature on board games. Silver et al. (Silver et al. 2018) addressed them in AlphaZero by combining MCTS with a learned value network and using $\gamma = 1$ (no discounting). Our results suggest that the combination of search and learning is not merely beneficial but may be *necessary* for effective play in adversarial games with sparse rewards.

Conclusion

We compared DQN, Double DQN, and MCTS in 5×5 Gardner Mini-Chess, a strategically rich but computationally manageable testbed for adversarial RL. Our experiments lead to three conclusions. First, MCTS with random rollouts substantially outperforms both DQN and Double DQN, demonstrating that explicit search provides a major advantage in deterministic games with sparse rewards. Second, Double DQN produces lower Q-value overestimation and more stable training compared to standard DQN, confirming the theoretical analysis of van Hasselt et al. (Van Hasselt, Guez, and Silver 2016) in a setting quite different from Atari. Third, both value-based methods converge to conservative, draw-heavy play, a consequence of sparse terminal rewards and the difficulty of credit assignment over long horizons.

These findings have practical implications. For practitioners applying DQN-family algorithms to adversarial domains, our results suggest that Double DQN should be preferred over standard DQN, not because it achieves dramatically higher win rates, but because it provides more stable and predictable training dynamics. More fundamentally, the gap between MCTS and the learned agents underscores the importance of search in game-playing AI, and motivates fu-

ture work on hybrid approaches.

Several limitations should be acknowledged. We used only two random seeds, limiting statistical power. The training budget of 20,000 episodes may be insufficient for full convergence. The Q-value clamping, while stabilizing, reduces the sensitivity of the overestimation analysis. Future work should explore reward shaping (intermediate rewards for captures, checks, or material advantage), undiscounted returns ($\gamma = 1$), prioritized experience replay, and the integration of learned value functions with MCTS in the style of AlphaZero.

References

- Bhatt, A. 2018. Reinforcement learning for Mini-Chess. *arXiv preprint arXiv:1812.06891*.
- Campbell, M.; Hoane Jr, A. J.; and Hsu, F.-h. 2002. Deep Blue. *Artificial Intelligence*, 134(1-2): 57–83.
- Coulom, R. 2006. Efficient selectivity and backup operators in Monte-Carlo tree search. In *International Conference on Computers and Games*, 72–83. Springer.
- Ehsan, M.; and Hashemi, S. H. 2012. Weakly solving Gardner Mini-Chess. *ICGA Journal*, 35(4): 215–221.
- Gardner, M. 1970. Mathematical games. *Scientific American*, 222(2): 104–109.
- Heinrich, J.; and Silver, D. 2016. Deep reinforcement learning from self-play in imperfect-information games. In *NIPS Deep Reinforcement Learning Workshop*.
- Kingma, D. P.; and Ba, J. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.
- Kocsis, L.; and Szepesvári, C. 2006. Bandit based Monte-Carlo planning. In *European Conference on Machine Learning*, 282–293. Springer.
- Lai, M. 2015. Giraffe: Using deep reinforcement learning to play chess. *arXiv preprint arXiv:1509.01549*.
- Lancot, M.; Lockhart, E.; Lepiau, J.-B.; Zambaldi, V.; Upadhyay, S.; Pérolat, J.; Srinivasan, S.; Timbers, F.; Tuyls, K.; Omidshafiei, S.; et al. 2019. OpenSpiel: A framework for reinforcement learning in games. *arXiv preprint arXiv:1908.09453*.
- Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A. A.; Veness, J.; Bellemare, M. G.; Graves, A.; Riedmiller, M.; Fidjeland, A. K.; Ostrovski, G.; et al. 2015. Human-level control through deep reinforcement learning. *Nature*, 518(7540): 529–533.
- Samuel, A. L. 1959. Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, 3(3): 210–229.
- Silver, D.; Huang, A.; Maddison, C. J.; Guez, A.; Sifre, L.; Van Den Driessche, G.; Schrittwieser, J.; Antonoglou, I.; Panneershelvam, V.; Lanctot, M.; et al. 2016. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587): 484–489.
- Silver, D.; Hubert, T.; Schrittwieser, J.; Antonoglou, I.; Lai, M.; Guez, A.; Lanctot, M.; Sifre, L.; Kumaran, D.; Graepel, T.; et al. 2018. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419): 1140–1144.

Tampuu, A.; Matiisen, T.; Kodelja, D.; Kuzovkin, I.; Korjus, K.; Aru, J.; Aru, J.; and Vicente, R. 2017. Multiagent cooperation and competition with deep reinforcement learning. *PloS one*, 12(4): e0172395.

Tian, Y.; and Zhu, Y. 2016. Better computer Go player with neural network and long-term prediction. In *International Conference on Learning Representations*.

Van Hasselt, H.; Guez, A.; and Silver, D. 2016. Deep reinforcement learning with double Q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 30.